

Технологии
Программирования

Волшебные палочки
Краткое руководство



Bad news

Экзамен. 28/10

Основа оценивания:

- Минимальный балл: 4,
- 3 вопроса по темам с лекций,
- Максимальный балл: 8.

4-й вопрос на 9 или 10:

- Тему выбираете самостоятельно,
- Вопрос будет задан по общим принципам, программирования по выбранной теме,
- Сложность вопроса можно выбрать (на 9 или на 10),
- От ответа можно отказаться без потери балла,
- Ошибка в ответе не понизит балл,
- Время на ответ ограничено (наугадить не получится).

В предыдущих сериях... Энергии

Имя - существительное

- Имя должно быть кратким и содержать информацию о том, что храниться в переменной

```
uint16_t m_color_rgb565 = 0x00FF; //переменная содержит цвет размером 2 байта в раскладке r5, g6, b5
```

Типы

- Использовать максимально говорящие типы, минимально необходимого размера.

```
uint8_t m_drawing_context = 0; //из типа видно, что контекстов не может быть больше 255
```

- Для сложных типов предпочитать структуры, стараться минимизировать использование анонимизированных таплов или пар.

RAII (*Resource Acquisition Is Initialization*)

- Переменная объявляется непосредственно перед использованием:

**ВСЕГДА ИНИЦИАЛИЗИРУЙТЕ ПЕРЕМЕННЫЕ СРАЗУ: В
МОМЕНТ ОБЪЯВЛЕНИЯ**



Основы заклинаний

Имя - Глагол

- Имя функции кратко описывает, что функция делает.

```
bool IsInitialized(const Object& object);  
uint32_t CombineValues(uint32_t base, uint32_t addon);  
void DrawLine(float x1, float y1, float x2, float y2);
```

Атомарность

- Делает именно то, что описано в названии, ничего больше,
- Содержит минимальное количество кода, необходимое для выполнения заложенной операции,
- В случае усложнения логики за счет сложных ветвлений, лучше рассмотреть возможность разбить ее,
- Использовать лямбды,
- Для удобства чтения, функция в идеале должна помещаться в экран,
- Минимизировать использование шаблонов,
- Минимизировать использование макросов.



Правильная форма



В определении следует заложить защиту от неправильного использования

- Contracts [C++20]
 - `[[expects: condition]]` for saying that an operation expects condition to hold for a call to complete successfully
 - `[[ensures: condition]]` for saying that an operation guarantees condition to hold after a successful call

```
bool CalculateOffset(size_t x) [[expects: x > 0]]; // требует чтобы параметр x был > 0
uint32_t GetBasicOffset(const Object& object) [[ensures r: r > 100]] // гарантирует что retval будет > 100
```

- Защита через сигнатуру. Защиту обеспечивает компилятор: специфицируется принцип работы с параметром, но не позволяет проверить значение.

```
bool CloneObject(const Object& source, Object& result); // оба параметра обязательны. source - in; result - inout
Object ConstructObject(const Parameter* options, Result* log); // оба параметра опциональны. options - in; log - inout
```

- Аннотирование параметров, самая слабая защита, всё на совести программиста:

```
Object ConstructObject(Parameter* in_options, Result* out_log);
```

Первый закон Мерфи:

Если существуют два способа сделать что-либо, и один из них ведёт к катастрофе, то кто-нибудь выберет именно этот способ

Виды

stdcall (cdecl)

- Обыкновенная C/C++ функция объявленная в программе, не принадлежит классу (кроме static функций).

thiscall

- Функция определенная внутри класса или структуры, может быть вызвана только с использованием объекта класса,

```
Object obj;  
obj.IsInitialized();
```

Lambda

- Неименованная функция, cdecl, которая объявляется внутри функции,
- Может использовать внешние переменные,
- Использует специальный синтаксис.

```
float reference = 24;  
auto myLambda = [=](float argument) -> bool { return argument > reference; };  
myLambda(12.f);
```

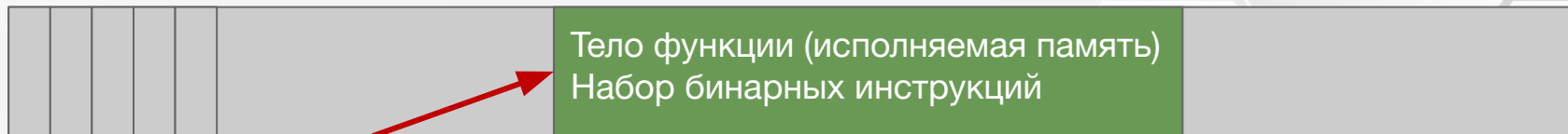
**Здесь использована нотация Microsoft.
Типов функций больше, но этих трех достаточно.**



Вопрос

Как выглядит функция изнутри?

Память



Указатель на начало исполняемого блока

Серцевина палочки

Каждая функция представлена указателем на область памяти

- Указатель на функцию подчиняется правилам работы с обыкновенными указателями.
- Можно сохранить в переменную и передавать как параметр,
- Вызов функции напрямую или по указателю осуществляется по одному правилу

```
// исходные функции:
Object ConstructObject(const Parameter* in_options, Result* out_log);
Object ConstructSystem(const Parameter* in_options, Result* out_log);

typedef Object(*objectFactory)(const Parameter*, Result*); // objectFactory - тип данных для функции

int main(int argc, char** argv)
{
    ConstructObject(nullptr, nullptr); // прямой вызов функции
    objectFactory standardConstructor = ConstructObject; // сохранение функции в переменную
    standardConstructor(nullptr, nullptr); // вызов функции сохраненной в переменную
    standardConstructor = ConstructSystem; // сохранение другой функции в переменную (внимание на сигнатуру)
    standardConstructor(nullptr, nullptr); // вызов
}
```

Так же можно было и в C.

Для получения указателя на thiscall используется технология bind



Простая

Ничего с C не поменялось.

- Функция состоит из 2х частей:
 - Заголовок (опционален):

```
bool    CloneObject (const Object& source, Object& result);  
|retval |имя        | блок параметров      |
```

- Тело:

```
bool    CloneObject (const Object& source, Object& result)  
{  
    // do something  
    return true;  
}
```

- Обе части могут находиться в разных файлах или в одном,
- Перед использованием функции она должна быть определена:
 - Заголовок объявлен выше по коду,
 - Тело объявлено выше по коду.



Близнецы



Классический C интерфейс

- Для каждого типа параметров создается своя функция, со своим названием

```
float absf(float fArgument); // функция для работы с float типами
int abs(int iArgument); // функция для работы с целочисленными параметрами
```

- Типичный представитель C интерфейса math.h и OpenGL, Vulkan
- Количество единиц интерфейса разрастается,
- Растет риск ошибки применения функции неверного типа...

Новшество C++ - переопределение функций (overload)

- Функции с разным набором параметров могут иметь одинаковое имя,
- Компилятор сам выберет какую функцию вызвать исходя из набора параметров,

```
float abs(float argument); // функция для работы с float типами
int abs(int argument); // функция для работы с целочисленными параметрами
double abs(float argument); // Егггг, необходимо переопределить параметр
```

- Если компилятор не сможет выбрать функцию, он выдаст ошибку о неоднозначности (ambiguity)
- Нельзя переопределить только возвращаемое значение.

Тёмная палочка

Lambda

- Неименованная функция*,
- Соответствует RAII, можно определить внутри функции, прямо перед использованием,
- Может использовать переменные внешней функции,
- Может быть вызвана сразу, при объявлении,

```
float reference = 24;
[=] (float argument) -> bool { return argument > reference; } (12.f);
захват | аргументы | retval | тело функции | вызов |
```

- Или отложено, в нужный момент, но для этого ее придется сохранить в переменную,

```
float reference = 24;
auto myLambda = [&] (float argument) -> bool { reference = 12; return argument > reference; };
переменная | захват | аргументы | retval | тело функции
myLambda(12.f); // отложенный вызов
```

- Механизм удобный для разового использования,
- Предикатные функции STL (find_if) - могут использовать лямбды, как предикаты.

***На самом деле это структура, называемая “функтор”, но если оно выглядит как функция, вызывается как функция, ведет себя как функция, пусть ей и будет.**



Приори инконтатем



Захват (Capture)

- [] - без захвата, в лямбде внешние переменные использовать нельзя,
- [=] - копирующий захват, копии всех внешних переменные можно использовать,
- [&] - все внешние переменные будут захвачены по ссылке, из лямбды можно менять их значение,
- Область захвата можно ограничить, только необходимым набором переменных.

```
float reference = 24;
int externalValueRef = 1234;
int externalValueCopy = 1234;

auto partialLambda = [&externalValueRef, externalValueCopy](int internalValue)
{
    externalValueRef = internalValue; // ок использование этой переменной разрешено в листе захвата
    reference = 5;                    // Еггог: в листе захвата этой переменной нет, пользоваться ей нельзя
};
```

**Внимание! Если переменная захвачена по ссылке,
то необходимо следить за ее временем жизни.**

Бузинная палочка

```
float abs(float argument); // функция для работы с float типами
int abs(int argument); // функция для работы с целочисленными параметрами
double abs(double argument); // функция для работы с double параметрами
word abs(word argument); // функция для работы с короткими целочисленными параметрами
```

Реализация у всех функций одинаковая! Что делать?

Ответ: Шаблоны.

- Универсальная реализация для всех типов вызова,
- Написали один раз и забыли.

```
template <typename T>
T templateAbs(T argument);
```

- Все правила работы с шаблонами здесь применимы
 - Несколько шаблонных аргументов,
 - Типы и значения, как аргументы шаблона,
 - Аргументы по умолчанию...

```
auto v1 = templateAbs(2.4f); // можно вызвать исходя из типа аргумента (как override)
auto v2 = templateAbs<uint32_t>(1.); // можно заставить вызвать нужную вам функцию (force)
```



Бузинная палочка

Частичная спецификация

```
template <typename T>
T templateAbs(T argument);
```

- Шаблонная функция реализует единый алгоритм для **ВСЕХ** типов,
- Определение функции формируется в момент ее вызова,
- Если в теле функции используется недопустимая для типа операция компилятор сообщит об ошибке,
- Если операция не имеет смысла, или требует дополнительных шагов или специального оператора, результат работы будет некорректным

```
struct Vector3D { float x = 0, y = 0 ,z = 0; }; // abs для этого типа бессмысленный
```

- Сделать override функцию для сложного типа: частичная спецификация

```
Vector3D templateAbs(Vector3D src)
{
    return {templateAbs(src.x), templateAbs(src.y), templateAbs(src.z)};
}
```

Компилятор будет выбирать функцию, сначала из частичных спецификаций, и только потом - конструировать новую по шаблону.



Палочка Рона

- Если функция может быть вызвана вне вашего модуля, она должна быть защищена дополнительно,
- Все параметры должны быть проверены на соответствие требованиям функции, (контракты),
- Возможность задать неверные параметры должна быть минимизирована, используйте компилятор (сигнатура),
- Используйте минимально необходимые типы параметров. Не надо делать параметр типа `int` если параметр должен быть ≥ 0 (`unsigned int` в помощь).

Если пользовательский ввод принять на веру, функция и весь модуль сработают как палочка вот у этого парня!



**Если функция может быть использована не вами, проверяйте ее параметры.
“НИКОГДА НЕ ДОВЕРЯЙ ПОЛЬЗОВАТЕЛЬСКОМУ ВВОДУ”**

Продолжение
следует

